ARRAYS – DATA STRUCTURES USING PYTHON

THE ARRAY DATA STRUCTURE

Definition

An **array** is a **collection of elements of the same data type** stored in **contiguous memory locations** and accessed using an **index**.

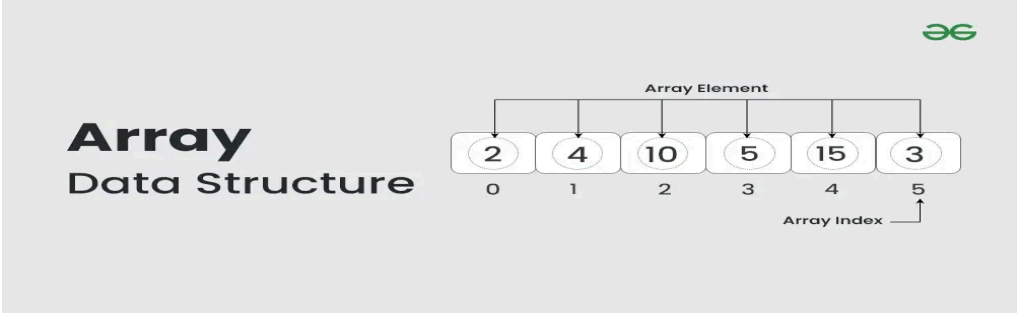
Key Features

- Fixed size
- Homogeneous elements
- Indexed access
- Stored in contiguous memory

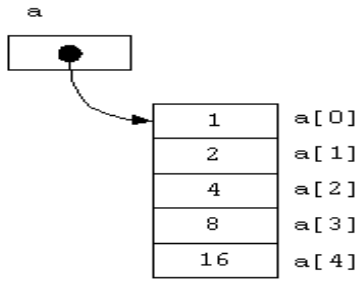
Array Representation

Example:

Index: 0 1 2 3 4
Array: 10 20 30 40 50



Pictographical Representation



Creation Of Array

- To create a NumPy array, you can use the function np.array(). • All you need to do to create a simple array is pass a list to it.
- Create an array with a range of elements: To create sequences of numbers, NumPy provides the arange function which is analogous to the Python built-in range, but returns an array.

```
IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct  4 2021, 19:00:18) [MSC v.1929 64 bit
D64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>import numpy as np
>>>a = np.array([1, 2, 3])
>>>a
array([1, 2, 3])
>>>
```

Example: 1d array

```
>>>import numpy as np
>>>a = np.arange(5)
>>>a
array([0, 1, 2, 3, 4])
>>>print(a)
[0 1 2 3 4]
```

2d array

```
>>>b = np.arange(12).reshape(4, 3) # 2d array
...
>>>b
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

3d array

```
c = np.arange(24).reshape(2, 3, 4)
c
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]])
```


Creating an array

Besides creating an array from a sequence of elements, you can easily create an array filled with 0's:

```
>>>np.zeros(2)
array([0., 0.])
>>>np.zeros((3, 4))
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

Creating an array

Create an empty array :

- The function empty creates an array whose initial content is random and depends on the state of the memory.
- The reason to use empty over zeros (or something similar) is speed - just make sure to fill every element afterwards!

```
>>>import numpy as np
>>> np.empty(2)
array([-4.03016855e-113,  1.22438294e+210])
```

Creating an array

An array filled with 1's:

```
>>> np.ones(2)
array([1., 1.])
>>> np.ones((2, 3, 4), dtype=np.int16)
array([[[1, 1, 1, 1],
        [1, 1, 1, 1],
        [1, 1, 1, 1]],

       [[1, 1, 1, 1],
        [1, 1, 1, 1],
        [1, 1, 1, 1]]], dtype=int16)
```

- numpy.reshape

Gives a new shape to an array without changing its data.

Syntax:

numpy.reshape(a, newshape, order='C')

Parameters:

a: array_like # Array to be reshaped.

newshape: int or tuple of ints

The newshape should be compatible with the original shape. If an integer, then the result will be a 1-D array of that length. One shape dimension can be -1. numpy allow us to give one of new shape parameter as -1 (eg: (2,-1) or (-1,3) but not (-1, -1)). It simply means that it is an unknown dimension and we want numpy to figure it out. And numpy will figure this by looking at the *'length of the array and remaining dimensions'* and making sure it satisfies the above mentioned criteria

```
>>> z = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
>>> z.shape
(3, 4)
>>> z.reshape(-1)
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
>>> z.reshape(-1,2)
array([[ 1,  2],
       [ 3,  4],
       [ 5,  6],
       [ 7,  8],
       [ 9, 10],
       [11, 12]])
>>> z.reshape(2, -1)
array([[ 1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12]])
```

Note: the reshape() does not changes the original array but only returns the changed array,

```
z.reshape(2, -1)
array([[ 1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12]])
z.shape
(3, 4)
z.reshape(4,3)
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [10, 11, 12]])
z.shape
(3, 4)
```


- If an array is too large to be printed, NumPy automatically skips the central part of the array and only prints the corners.
- Arithmetic operators on arrays apply elementwise.
- A new array is created and filled with the result.

```
>>> a=np.array([10, 20, 30, 40])
>>> a
array([10, 20, 30, 40])
>>> b = np.arange(4)
>>> b
array([0, 1, 2, 3])
>>> c=a-b
>>> c
array([10, 19, 28, 37])
>>> d=a+b
>>> d
array([10, 21, 32, 43])
```

```
print(np.arange(10000))
[   0    1    2 ... 9997 9998 9999]
print(np.arange(10000).reshape(100, 100))
[[   0    1    2 ...   97   98   99]
 [ 100  101  102 ...  197  198  199]
 [ 200  201  202 ...  297  298  299]
 ...
 [9700 9701 9702 ... 9797 9798 9799]
 [9800 9801 9802 ... 9897 9898 9899]
 [9900 9901 9902 ... 9997 9998 9999]]
```

Types of Arrays

1. One-Dimensional Array
2. Two-Dimensional Array
3. Multidimensional Array **Creating an Array in Python**

arr = [10, 20, 30, 40, 50]

To create and display an array.

```
# PROGRAM 1: ARRAY DATA STRUCTURE

arr = [10, 20, 30, 40, 50]

print("Array Elements:")
for i in arr:
    print(i, end=" ")
```

✅ **Output**

```
Array Elements:
10 20 30 40 50
```

OPERATIONS ON ARRAYS

1. Traversal: Accessing and processing each element of the array.

Algorithm

1. Start from index 0
2. Visit each element till end

Python Code

```
for i in arr:
    print(i)
```

2. Insertion: Adding an element at a specific position.

Example

Insert 25 at index 2

Algorithm

1. Shift elements to the right
2. Insert new element

Python Code

```
arr.insert(2, 25)
```

3. Deletion: Removing an element from the array.

Algorithm

1. Locate element

2. Shift remaining elements left

Python Code

```
arr.remove(30)
```

4. Searching: Finding the position of an element.

Python Code

```
if 40 in arr:
    print("Found")
```

5. Updating: Changing an element value.

Python Code

```
arr[1] = 100
```

OPERATIONS ON ARRAYS

```
arr = [10, 20, 30, 40, 50]
print("Original Array:", arr)
# Traversal
print("\nTraversal:")
for i in arr:
    print(i, end=" ")
# Insertion
arr.insert(2, 25)
print("\n\nAfter Insertion:", arr)

# Deletion
arr.remove(30)
print("After Deletion:", arr)

# Searching
search = 40
if search in arr:
    print("Element", search, "found at index", arr.index(search))
else:
    print("Element not found")

# Updating
arr[1] = 100
print("After Updating:", arr)
```

✅ Output

yaml

Copy code

```
Original Array: [10, 20, 30, 40, 50]
Traversal:
10 20 30 40 50
```

After Insertion: [10, 20, 25, 30, 40, 50]
After Deletion: [10, 20, 25, 40, 50]
Element 40 found at index 3
After Updating: [10, 100, 25, 40, 50]

TWO-DIMENSIONAL ARRAYS (2D ARRAYS)

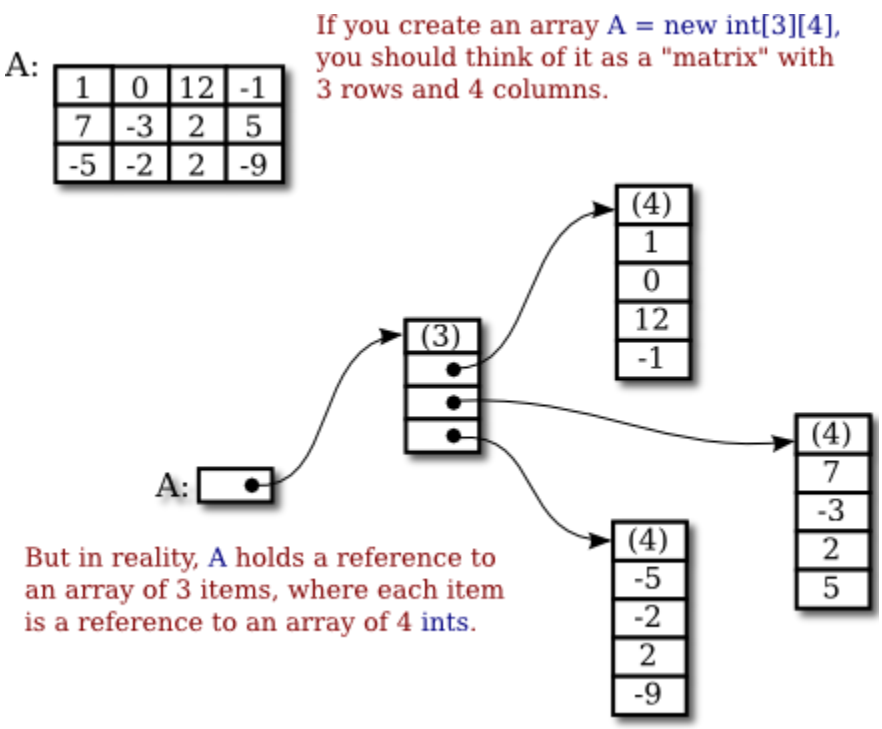
A **2D array** is an array of arrays arranged in **rows and columns**, similar to a **matrix**.

Representation

Matrix A = 3 × 3

10 20 30
40 50 60
70 80 90

Pictographical Representation



Memory Representation (Row-Major Order)

Elements are stored row by row in memory.

Creating a 2D Array in Python

```
matrix = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]
```

Accessing Elements in 2D Array

```
print(matrix[1][2])  # Output: 60
```

Traversing a 2D Array

Algorithm

1. Loop through rows
2. Loop through columns

Python Code

```
for i in range(len(matrix)):
    for j in range(len(matrix[0])):
        print(matrix[i][j], end=" ")
    print()
```

2D Array Operations

Matrix Addition

```
A = [[1,2],[3,4]]
B = [[5,6],[7,8]]

C = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]
print(C)
```

Matrix Transpose

```
transpose = [[matrix[j][i] for j in range(len(matrix))]
             for i in range(len(matrix[0]))]
print(transpose)
```

Advantages of Arrays

✔ Fast access
✔ Simple structure
✔ Efficient memory usage

Disadvantages of Arrays

✖ Fixed size
✖ Insertion & deletion costly
✖ Wastage of memory possible

To create and display a two-dimensional array (matrix).

TWO DIMENSIONAL ARRAY

```
matrix = [
```

```
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print("Two Dimensional Array:")
for i in range(len(matrix)):
    for j in range(len(matrix[0])):
        print(matrix[i][j], end=" ")
    print()
```

✅ **Output**

```
Two Dimensional Array:
1 2 3
4 5 6
7 8 9
```

LINKED STRUCTURES

A data structure can be defined as follows...

Data structure is a method of organizing a large amount of data more efficiently so that any operation on that data becomes easy

Based on the organizing method of data structure, data structures are divided into two types.

- Linear Data Structures
- Non - Linear Data Structures

Linear Data Structures

If a data structure organizes the data in sequential order, then that data structure is called a Linear Data Structure.

Example

1. Arrays
2. List (Linked List)
3. Stack
4. Queue

Non - Linear Data Structures

If a data structure organizes the data in random order, then that data structure is called a Non-Linear Data Structure.

Example

1. Tree
2. Graph
3. Dictionaries
4. Heaps
5. Tries, Etc.

Linked List

When we want to work with an unknown number of data values, we use a linked list data structure to organize that data. The linked list is a linear data structure that contains a sequence of elements such that each element links to its next element in the sequence. Each element in a linked list is called "Node".

Single Linked List

Simply a list is a sequence of data, and the linked list is a sequence of data linked with each other.

The formal definition of a single linked list is as follows...
A single linked list is a sequence of elements in which every element has a link to its next element in the sequence.
In any single linked list, the individual element is called a "Node". Every "Node" contains two fields, a data field, and the next field. The data field is used to store the actual value of the node and the next field is used to store the address of the next node in the sequence. The graphical representation of a node in a single linked list is as follows...

Operations on Single Linked List
The following operations are performed on a Single Linked List
Insertion
Deletion
Display
Before we implement actual operations, first we need to set up an empty list. First, perform the following steps before implementing actual operations.
Step 1 - Include all the header files which are used in the program.
Step 2 - Declare all the user defined functions.
Step 3 - Define a Node structure with two members data and next
Step 4 - Define a Node pointer 'head' and set it to NULL.
Step 5 - Implement the main method by displaying the operations menu and make suitable function calls in the main method to perform user selected operation.

Insertion
In a single linked list, the insertion operation can be performed in three ways. They are as follows...
1. Inserting At Beginning of the list
2. Inserting At End of the list
3. Inserting At Specific location in the list

Inserting At Beginning of the list
We can use the following steps to insert a new node at the beginning of the single linked list...
Step 1 - Create a newNode with a given value.
Step 2 - Check whether list is Empty (head == NULL)
Step 3 - If it is Empty then, set newNode→next = NULL and head = newNode.
Step 4 - If it is Not Empty then, set newNode→next = head and head = newNode.

Inserting At End of the list
We can use the following steps to insert a new node at the end of the single linked list...
Step 1 - Create a newNode with given value and newNode → next as NULL.
Step 2 - Check whether the list is Empty (head == NULL).
Step 3 - If it is Empty then, set head = newNode.
Step 4 - If it is Not Empty then, define a node pointer temp and initialize with head.
Step 5 - Keep moving the temp to its next node until it reaches the last node in the list (until temp → next is equal to NULL).
Step 6 - Set temp → next = newNode.

Inserting At Specific location in the list (After a Node)
We can use the following steps to insert a new node after a node in the single linked list...

Step 1 - Create a newNode with a given value.
Step 2 - Check whether list is Empty (head == NULL)
Step 3 - If it is Empty then, set newNode → next = NULL and head = newNode.
Step 4 - If it is Not Empty then, define a node pointer temp and initialize with head.
Step 5 - Keep moving the temp to its next node until it reaches to the node after which we want to insert the newNode
(until temp1 → data is equal to location, here location is the node value after which we want to insert the newNode).
Step 6 - Every time check whether temp has reached the last node or not. If it is reached to last node then display 'Given
node is not found in the list!!! Insertion is not possible!!!' and terminate the function. Otherwise move the temp to next
node.
Step 7 - Finally, Set 'newNode → next = temp → next' and 'temp → next = newNode'

Deletion

In a single linked list, the deletion operation can be performed in three ways. They are as follows...
1. Deleting from Beginning of the list
2. Deleting from End of the list
3. Deleting a Specific Node

Deleting from Beginning of the list

We can use the following steps to delete a node from the beginning of the single linked list...
Step 1 - Check whether list is Empty (head == NULL)
Step 2 - If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminates the function.
Step 3 - If it is Not Empty then, define a Node pointer 'temp' and initialize with head.
Step 4 - Check whether list is having only one node (temp → next == NULL)
Step 5 - If it is TRUE then set head = NULL and delete temp (Setting Empty list conditions)
Step 6 - If it is FALSE then set head = temp → next, and delete temp.

Deleting from End of the list

We can use the following steps to delete a node from the end of the single linked list...
Step 1 - Check whether list is Empty (head == NULL)
Step 2 - If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminates the function.
Step 3 - If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.
Step 4 - Check whether list has only one Node (temp1 → next == NULL)
Step 5 - If it is TRUE. Then, set head = NULL and delete temp1. And terminate the function. (Setting Empty list
condition)
Step 6 - If it is FALSE. Then, set 'temp2 = temp1 ' and move temp1 to its next node. Repeat the same until it reaches
to the last node in the list. (until temp1 → next == NULL)
Step 7 - Finally, Set temp2 → next = NULL and delete temp1.

Deleting a Specific Node from the list

We can use the following steps to delete a specific node from the single linked list...
Step 1 - Check whether list is Empty (head == NULL)

Step 2 - If it is Empty then, display 'List is Empty!!! Deletion is not possible' and terminates the function.
Step 3 - If it is Not Empty then, define two Node pointers 'temp1' and 'temp2' and initialize 'temp1' with head.
Step 4 - Keep moving the temp1 until it reaches the exact node to be deleted or to the last node. And every time set 'temp2 = temp1' before moving the 'temp1' to its next node.
Step 5 - If it is reached to the last node then display 'Given node not found in the list! Deletion is not possible!!!'. And terminate the function.
Step 6 - If it is reached to the exact node which we want to delete, then check whether list is having only one node or not
Step 7 - If the list has only one node and that is the node to be deleted, then set head = NULL and delete temp1 (free(temp1)).
Step 8 - If the list contains multiple nodes, then check whether temp1 is the first node in the list (temp1 == head).
Step 9 - If temp1 is the first node then move the head to the next node (head = head → next) and delete temp1.
Step 10 - If temp1 is not the first node then check whether it is the last node in the list (temp1 → next == NULL).
Step 11 - If temp1 is the last node then set temp2 → next = NULL and delete temp1 (free(temp1)).
Step 12 - If temp1 is not first node and not last node then set temp2 → next = temp1 → next and delete temp1 (free(temp1)).
Displaying a Single Linked List
We can use the following steps to display the elements of a single linked list...
Step 1 - Check whether list is Empty (head == NULL)
Step 2 - If it is Empty then, display 'List is Empty!!!' and terminate the function.
Step 3 - If it is Not Empty then, define a Node pointer 'temp' and initialize with head.
Step 4 - Keep displaying temp → data with an arrow (--->) until temp reaches to the last node
Step 5 - Finally display temp → data with an arrow pointing to NULL (temp → data ---> NULL).

Define Node structure

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

- ## Define head pointer

```
head = None
```

- ## Step 5 – Implement Linked List Operations

- **Insertion at Beginning**

```
def insert_at_beginning(value):
    global head
    newNode = Node(value)
```

```
    if head is None:
        newNode.next = None
        head = newNode
    else:
        newNode.next = head
        head = newNode
```

- **Insertion at End**

```
def insert_at_end(value):
    global head
    newNode = Node(value)
    newNode.next = None

    if head is None:
        head = newNode
    else:
        temp = head
        while temp.next is not None:
            temp = temp.next
        temp.next = newNode
```

- **Insertion After a Specific Node**

```
def insert_after_node(location, value):
    global head
    newNode = Node(value)

    if head is None:
        print("List is Empty! Insertion not possible.")
        return

    temp = head
    while temp is not None:
        if temp.data == location:
            newNode.next = temp.next
            temp.next = newNode
            return
        temp = temp.next

    print("Given node is not found in the list!!! Insertion is not possible!!!")
```

🔹 Deletion Operations

🔸 Deleting from Beginning

```
def delete_from_beginning():
    global head

    if head is None:
        print("List is Empty!!! Deletion is not possible")
        return

    temp = head
    head = head.next
    del temp
```

🔸 Deleting from End

```
def delete_from_end():
    global head

    if head is None:
        print("List is Empty!!! Deletion is not possible")
        return

    if head.next is None:
        temp = head
        head = None
        del temp
        return

    temp1 = head
    temp2 = None

    while temp1.next is not None:
        temp2 = temp1
        temp1 = temp1.next

    temp2.next = None
```

```
    del temp1
```

🔸 Deleting a Specific Node

```
def delete_specific_node(value):
    global head

    if head is None:
        print("List is Empty!!! Deletion is not possible")
        return

    temp1 = head
    temp2 = None

    while temp1 is not None and temp1.data != value:
        temp2 = temp1
        temp1 = temp1.next

    if temp1 is None:
             print("Given  node  not  found  in  the  list!  Deletion  is  not
possible!!!")
        return

    if temp1 == head:
        head = head.next
        del temp1
    elif temp1.next is None:
        temp2.next = None
        del temp1
    else:
        temp2.next = temp1.next
        del temp1
```

🔹 Display Linked List

```
def display():
    if head is None:
        print("List is Empty!!!")
        return
```

```
    temp = head
    while temp.next is not None:
        print(temp.data, end=" ---> ")
        temp = temp.next

    print(temp.data, "---> NULL")
```

🔹 Main Menu-Driven Program

```
def main():
    while True:
        print("\n----- SINGLE LINKED LIST OPERATIONS -----")
        print("1. Insert at Beginning")
        print("2. Insert at End")
        print("3. Insert After a Node")
        print("4. Delete from Beginning")
        print("5. Delete from End")
        print("6. Delete a Specific Node")
        print("7. Display List")
        print("8. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            value = int(input("Enter value: "))
            insert_at_beginning(value)

        elif choice == 2:
            value = int(input("Enter value: "))
            insert_at_end(value)

        elif choice == 3:
            loc = int(input("Enter location value: "))
            value = int(input("Enter new value: "))
            insert_after_node(loc, value)

        elif choice == 4:
            delete_from_beginning()
```

```
        elif choice == 5:
            delete_from_end()

        elif choice == 6:
            value = int(input("Enter value to delete: "))
            delete_specific_node(value)

        elif choice == 7:
            display()

        elif choice == 8:
            print("Exiting Program...")
            break

        else:
            print("Invalid choice! Try again.")
```

🔹 Run the Program

```
main()
```

✅ Sample Output

```
10 ---> 20 ---> 30 ---> NULL
```

INTERFACES: DEVELOPING AN INTERFACE

What is an Interface?

An interface is a blueprint that defines what methods a class must implement, but not how they are implemented.

In Python, interfaces are developed using the `abc` (Abstract Base Class) module.

Why Interfaces are Needed in Data Structures?

Interfaces help to:

- Enforce standard operations (insert, delete, display)
- Provide abstraction
- Ensure uniform behavior across different data structures
- Improve code reusability and flexibility

Interface in Python

Python does not have a keyword called `interface`.
Instead, it uses:

- Abstract Base Classes (ABC)
- @abstractmethod decorator

Steps to Develop an Interface in Python

Step 1 – Import ABC module
Step 2 – Create an abstract class
Step 3 – Declare abstract methods
Step 4 – Implement the interface in a concrete class

🔹 Syntax: Developing an Interface in Python

```
from abc import ABC, abstractmethod
class InterfaceName(ABC):
    @abstractmethod
    def method1(self):
        pass
    @abstractmethod
    def method2(self):
        pass
```

Example: Interface for Data Structure Operations

🔸 Step 1 – Develop Interface

```
from abc import ABC, abstractmethod
class DataStructure(ABC):
    @abstractmethod
    def insert(self):
        pass
    @abstractmethod
    def delete(self):
        pass
```

```
    @abstractmethod
    def display(self):
        pass
```

Step 2 – Implement Interface (Linked List Example)

```
class LinkedList(DataStructure):

    def __init__(self):
        self.list = []

    def insert(self, value):
        self.list.append(value)
        print("Inserted:", value)

    def delete(self, value):
        if value in self.list:
            self.list.remove(value)
            print("Deleted:", value)
        else:
            print("Element not found")

    def display(self):
        print("Linked List:", self.list)
```

Step 3 – Use Interface Object

```
obj = LinkedList()

obj.insert(10)
obj.insert(20)
obj.insert(30)
obj.display()

obj.delete(20)
obj.display()
```

🔹 Output

```
Inserted: 10
Inserted: 20
```

```
Inserted: 30
Linked List: [10, 20, 30]
Deleted: 20
Linked List: [10, 30]
```

🔹 Important Rules of Interface in Python

- **Cannot create object of interface directly**
- **All abstract methods must be implemented**
- **Supports multiple inheritance**
- **Enforces method consistency**

🔹 Advantages of Interfaces

- **Achieves abstraction**
- **Improves code structure**
- **Easy to extend data structures**
- **Promotes standardization**